

Semana 6
ISIS-1611: Inteligencia Artificial

Carla González

11 de julio de 2026

1. Agentes Lógicos II: Inferencia Proposicional

1.1. Reglas de Inferencia

Regla	Forma
Modus Ponens	$\alpha \Rightarrow \beta, \alpha \vdash \beta$
Modus Tollens	$\alpha \Rightarrow \beta, \neg\beta \vdash \neg\alpha$
Resolución	$(\ell_1 \vee \dots \vee \ell_k), (\neg\ell_1 \vee m_1 \vee \dots) \vdash (m_1 \vee \dots)$

1.2. Resolución

Algoritmo de inferencia completo para lógica proposicional en CNF:

1. Convertir $KB \wedge \neg\alpha$ a CNF.
2. Aplicar resolución hasta obtener la cláusula vacía (contradicción) o no poder seguir.
3. Si se obtiene contradicción $\Rightarrow KB \models \alpha$.

1.3. DPLL y WalkSAT

- **DPLL**: backtracking completo sobre asignaciones de verdad, con propagación de cláusulas unitarias.
- **WalkSAT**: búsqueda local incompleta, eficaz en la práctica para instancias satisfacibles.

1.4. Ejemplo: Refutación por Resolución

Ejemplo 1.1 (Refutación). Sea $KB = \{P, P \Rightarrow Q, Q \Rightarrow R\}$ y se quiere probar $KB \models R$. Se convierte $KB \wedge \neg R$ a CNF: $\{P, \neg P \vee Q, \neg Q \vee R, \neg R\}$, y se aplica resolución hasta obtener la cláusula vacía $\square$ (una contradicción), lo que confirma el entailment.

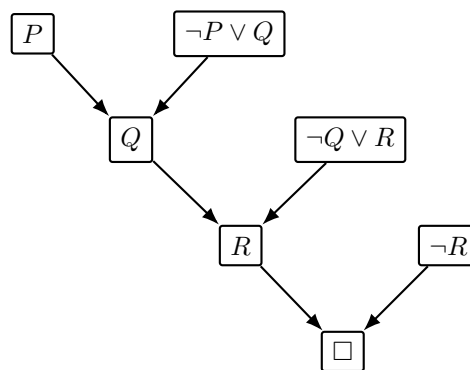


Figura 1: Árbol de refutación por resolución: cada nodo se obtiene combinando los dos que apuntan hacia él; llegar a la cláusula vacía $\square$ demuestra que $KB \wedge \neg R$ es insatisfacible, es decir, $KB \models R$.

Ejercicio 1.1. Sea $KB = \{A \vee B, \neg A \vee C, \neg B \vee C, \neg C \vee D\}$. Se quiere probar $KB \models D$ por refutación.

1. Escriba la cláusula que se agrega a KB al negar la meta.
2. Aplique resolución sobre $A \vee B$ y $\neg A \vee C$, y por separado sobre $\neg B \vee C$, para derivar una cláusula con únicamente C .

3. Combine el resultado anterior con $\neg C \vee D$ y con la meta negada para llegar a la cláusula vacía.
4. Dibuje el árbol de refutación completo, análogo al de la Figura anterior.

2. Lógica de Primer Orden (LPO)

2.1. Sintaxis

Elementos de LPO:

- **Constantes:** *Juan, UNIANDES, ...*
- **Variables:** *x, y, z, ...*
- **Predicados:** *Estudiante(x), Mayor(x, y)*
- **Funciones:** *Padre(x), Suma(x, y)*
- **Cuantificadores:** $\forall$ (universal), $\exists$ (existencial)
- **Conectivos:** $\neg, \wedge, \vee, \Rightarrow, \Leftrightarrow$

2.2. Semántica

Un **modelo** en LPO especifica:

- El dominio de objetos
- La interpretación de constantes, predicados y funciones

2.3. Cuantificadores

$$\forall x P(x) \equiv \neg \exists x \neg P(x)$$

$$\exists x P(x) \equiv \neg \forall x \neg P(x)$$

Nota 2.1. $\forall x (P(x) \Rightarrow Q(x))$ es diferente de $\forall x (P(x) \wedge Q(x))$. El cuantificador universal normalmente usa $\Rightarrow$; el existencial usa $\wedge$.

2.4. Ejemplo: Modelo en LPO

Ejemplo 2.1 (Modelo simple). Sea el dominio $D = \{\text{Juan, Maria, Pedro}\}$ y el predicado binario $\text{Padre}(x, y)$ (*x es padre de y*). La interpretación de la Figura siguiente satisface $\text{Padre}(\text{Juan, Maria})$ y $\text{Padre}(\text{Juan, Pedro})$, y ninguna otra pareja.

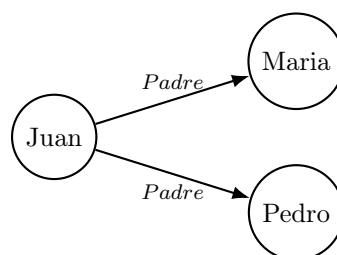


Figura 2: Modelo con dominio $\{\text{Juan, Maria, Pedro}\}$: cada flecha es una pareja (x, y) para la cual $\text{Padre}(x, y)$ es verdadero en este modelo. Cambiar las flechas cambia el modelo sin cambiar la sintaxis de la oración.

Ejercicio 2.1. Usando los predicados $Estudiante(x)$, $Curso(y)$, $Aprueba(x, y)$ y $Certificado(x, y)$, traduzca a LPO:

1. Todo estudiante que aprueba un curso recibe un certificado de ese curso.
2. Existe al menos un estudiante que no ha aprobado ningún curso.
3. Ningún estudiante aprueba todos los cursos.

Para cada oración, indique si el cuantificador principal es $\forall$ o $\exists$ y justifique si el conectivo correcto dentro de su alcance es $\Rightarrow$ o $\wedge$.

3. LPO e Inferencia I

3.1. Bases de Conocimiento en LPO

Las bases de conocimiento en LPO pueden expresar relaciones complejas de forma compacta, a diferencia de la lógica proposicional.

3.2. Unificación

Para aplicar reglas de inferencia en LPO se necesita **unificar** expresiones:

$$\text{UNIFY}(\alpha, \beta) = \theta \text{ tal que } \text{SUBST}(\theta, \alpha) = \text{SUBST}(\theta, \beta)$$

Algoritmo: recorre la estructura de ambas expresiones, construyendo sustituciones. Devuelve *fallo* si no hay unificador.

3.3. Inferencia con Cuantificadores

- **Instanciación universal:** $\forall x \alpha(x) \vdash \alpha(c)$ para cualquier constante c .
- **Instanciación existencial:** $\exists x \alpha(x) \vdash \alpha(k)$ donde k es una nueva constante de Skolem.
- **Skolemización:** eliminar cuantificadores existenciales introduciendo constantes/funciones de Skolem.

3.4. Forma Normal de Clausura (CNF en LPO)

Pasos para convertir a CNF:

1. Eliminar bicondicionales e implicaciones.
2. Mover $\neg$ hacia adentro (De Morgan, reglas de cuantificadores).
3. Estandarizar variables.
4. Skolemizar.
5. Eliminar cuantificadores universales.
6. Distribuir $\vee$ sobre $\wedge$.

3.5. Ejemplo: Unificación

Ejemplo 3.1 (Unificación de dos átomos). Para unificar $Padre(x, Juan)$ con $Padre(Pedro, y)$, el algoritmo recorre ambas expresiones posición por posición: en la primera posición x debe sustituirse por Pedro; en la segunda, y debe sustituirse por Juan. El unificador más general es $\theta = \{x/Pedro, y/Juan\}$.

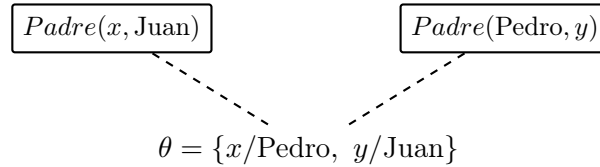


Figura 3: Unificación de $Padre(x, Juan)$ y $Padre(Pedro, y)$: aplicar θ a ambas expresiones produce la misma expresión, $Padre(Pedro, Juan)$.

- Ejercicio 3.1.**
1. Encuentre, si existe, el unificador más general de $Quiere(x, Helado(x))$ y $Quiere(Juan, Helado(Vainilla))$.
 2. Encuentre el unificador de $Hermano(x, y)$ y $Hermano(Juan, y)$. ¿Por qué es necesario estandarizar variables antes de unificar dos cláusulas que provienen de la misma regla?
 3. Convierta a CNF: $\forall x (Estudiante(x) \Rightarrow \exists y (Curso(y) \wedge Inscrito(x, y)))$, indicando en qué paso aparece la función de Skolem y qué representa.

4. Inferencia en LPO II

4.1. Encadenamiento hacia adelante (Forward Chaining)

Aplica reglas de la forma $P_1 \wedge \dots \wedge P_n \Rightarrow Q$ para derivar nuevos hechos hasta alcanzar la meta.

- Completo para bases de conocimiento de **cláusulas de Horn**.
- Dirigido por los datos.

4.2. Encadenamiento hacia atrás (Backward Chaining)

Parte de la meta y trabaja hacia atrás buscando hechos o submetas.

- Completo para cláusulas de Horn.
- Base de Prolog.
- Puede ciclar si no se controla.

4.3. Resolución en LPO

Generaliza la resolución proposicional usando unificación:

$$\frac{\ell_1 \vee \dots \vee \ell_k \quad \neg m_1 \vee \dots \vee \neg m_j}{\text{SUBST}(\theta, \ell_1 \vee \dots \vee m_j)}$$

donde $\theta = \text{UNIFY}(\ell_i, m_k)$.

- **Refutación por resolución**: completa para LPO (Teorema de Completitud de Gödel).

4.4. Ejemplo: El caso de Nono (¿Es West un criminal?)

Ejemplo 4.1 (Base de conocimiento). 1. $American(x) \wedge Weapon(y) \wedge Sells(x, y, z) \wedge Hostile(z) \Rightarrow Criminal(x)$

2. $Owns(Nono, M1)$

3. $Missile(M1)$

4. $Missile(x) \wedge Owns(Nono, x) \Rightarrow Sells(West, x, Nono)$

5. $Missile(x) \Rightarrow Weapon(x)$

6. $Enemy(x, America) \Rightarrow Hostile(x)$

7. $American(West)$

8. $Enemy(Nono, America)$

Meta: probar $Criminal(West)$.

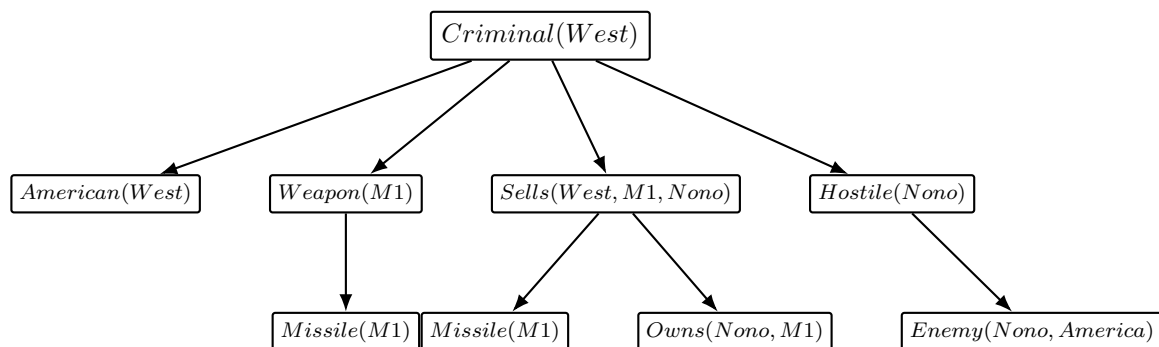


Figura 4: Árbol de encadenamiento hacia atrás para $Criminal(West)$: los cuatro hijos de la raíz son las **cuatro submetas** que exige la conjunción de la regla 1 (todas deben probarse), y $Missile(M1)$ y $Owns(Nono, M1)$ son las dos submetas que exige la regla 4. Cada hoja es un hecho que ya está en la KB (reglas 2, 3, 7 y 8).

Submeta	Regla utilizada	Qué se obtiene
$Criminal(West)$	Regla 1	Se descompone en 4 submetas (AND)
$American(West)$	Regla 7 (hecho)	Submeta resuelta directamente
$Weapon(M1)$	Regla 5	Se reduce a la submeta $Missile(M1)$
$Missile(M1)$	Regla 3 (hecho)	Submeta resuelta directamente
$Sells(West, M1, Nono)$	Regla 4	Se reduce a $Missile(M1) \wedge Owns(Nono, M1)$
$Owns(Nono, M1)$	Regla 2 (hecho)	Submeta resuelta directamente
$Hostile(Nono)$	Regla 6	Se reduce a $Enemy(Nono, America)$
$Enemy(Nono, America)$	Regla 8 (hecho)	Submeta resuelta directamente

Cuadro 1: Traza del encadenamiento hacia atrás: en cada fila se indica qué regla dispara la reducción y qué submeta nueva (o hecho) se obtiene, hasta que todas las hojas quedan probadas y $Criminal(West)$ queda demostrado.

Ejercicio 4.1. 1. Usando la misma KB, construya la traza de **encadenamiento hacia adelante**: parta de los hechos (reglas 2, 3, 7, 8) y muestre, en una tabla como la anterior, en qué orden se disparan las reglas 5, 6, 4 y 1 hasta derivar $Criminal(West)$.

2. Compare las dos trazas: ¿por qué el encadenamiento hacia atrás solo explora los hechos relevantes para la meta, mientras que el hacia adelante puede derivar hechos que nunca se usan?
3. Dado el hecho adicional $Enemy(Nono, UK)$ (en vez de $America$), ¿sigue siendo posible probar $Criminal(West)$ con esta KB? Justifique cuál regla falla.

Ejercicio 4.2 (Resolución en LPO). Sean las cláusulas

$$\neg Ama(x, Padre(x)) \vee Feliz(x) \quad \text{y} \quad Ama(Juan, Padre(Juan)).$$

1. Encuentre el unificador θ entre los literales complementarios.
2. Aplique $SUBST(\theta, \cdot)$ a la cláusula resolvente resultante.
3. ¿Qué hecho queda demostrado?

5. Ejercicios: Guía de Agentes Lógicos (con solución paso a paso)

Esta sección resuelve, de principio a fin, la *Guía de Ejercicios: Agentes Lógicos*. Cada respuesta se justifica explícitamente; en los casos falsos se da la versión corregida, y en los casos de tablas de verdad o resolución se muestra cada paso.

5.1. Bloque 1: Conceptos Fundamentales de Agentes Lógicos

Parte A — Consecuencia Lógica y Modelos

1. “Una base de conocimiento KB implica lógicamente una sentencia α (escrito $KB \models \alpha$) si y solo si α es verdadera en todo modelo en el que KB es verdadera.”
Verdadero. Es exactamente la definición semántica de consecuencia lógica: un modelo de KB es una interpretación donde toda sentencia de KB es verdadera, y $KB \models \alpha$ exige que α también lo sea en *todos* esos modelos.
2. “Si $KB \models \alpha$, entonces α debe ser verdadera en todos los mundos posibles, sin importar lo que diga KB .”
Falso. α solo tiene que ser verdadera en los modelos donde KB también es verdadera; en los modelos donde KB es falsa, α puede ser falsa sin violar $KB \models \alpha$.
Corrección: Si $KB \models \alpha$, entonces α es verdadera en todo modelo en el que KB es verdadera (no en todo modelo posible sin restricción).
3. “Una sentencia es válida (tautología) si es verdadera en algunos los modelos posibles.”
Falso. Eso describe *satisfacibilidad*, no validez.
Corrección: Una sentencia es válida (tautología) si es verdadera en **todos** los modelos posibles.
4. “Una sentencia es satisfacible si existe al menos un modelo en el que es verdadera.”
Verdadero. Es la definición estándar de satisfacibilidad.
5. “Si una sentencia es insatisfacible, entonces su negación es satisfacible.”
Verdadero. Si α es insatisfacible, es falsa en todo modelo, por lo tanto $\neg\alpha$ es verdadera en todo modelo (es decir, $\neg\alpha$ es válida). Toda sentencia válida es, en particular, satisfacible (verdadera en al menos un modelo: todos ellos).

Parte B — Algoritmos de Inferencia y Verificación de Modelos

1. “El algoritmo *TT-Implica?* enumera todos los modelos posibles para verificar si $KB \models \alpha$, siendo correcto y completo pero exponencial en el número de símbolos.”
Verdadero. Con n símbolos proposicionales hay 2^n asignaciones de verdad posibles; *TT-Implica?* las recorre todas, por lo que es $\mathcal{O}(2^n)$, pero al ser una enumeración exhaustiva es correcto (sound) y completo.
2. “Un procedimiento de prueba es correcto (sound) si sólo deriva sentencias que son consecuencia lógica de KB .”
Verdadero. Esa es la definición de *soundness*: nunca deriva una sentencia falsa a partir de premisas verdaderas. (La *completitud* es la propiedad complementaria: deriva *todas* las sentencias que son consecuencia lógica.)
3. “Modus Ponens establece: si $\alpha \Rightarrow \beta$ está en KB , y β es verdadera, entonces podemos inferir α .”
Falso. Esto es la falacia de *afirmar el consecuente*, una regla inválida.
Corrección: Modus Ponens establece que si $\alpha \Rightarrow \beta$ está en KB y α es verdadera, entonces podemos inferir β .
4. “La regla de resolución establece que de las cláusulas $(\ell_1 \vee \dots \vee \ell_k \vee m)$ y $(\neg m \vee n_1 \vee \dots \vee n_j)$ se puede derivar $(\ell_1 \vee \dots \vee \ell_k \vee n_1 \vee \dots \vee n_j)$.”
Verdadero. Es exactamente la regla de resolución binaria: se cancela el par complementario $m/\neg m$ y se unen el resto de los literales de ambas cláusulas.
5. “El algoritmo *DPLL* es una mejora sobre la enumeración básica de tablas de verdad para verificar satisfacibilidad.”
Verdadero. *DPLL* añade propagación de cláusulas unitarias, eliminación de literales puros y poda temprana (*early termination*) sobre el backtracking de fuerza bruta, evitando explorar ramas completas del árbol de asignaciones cuando ya se sabe que fallarán.
6. “El encadenamiento hacia adelante está orientado a metas: trabaja hacia atrás desde una consulta para determinar si KB la soporta.”
Falso. Esa descripción corresponde al encadenamiento **hacia atrás**.
Corrección: El encadenamiento hacia adelante está orientado a **datos**: parte de los hechos conocidos en KB y dispara reglas repetidamente hasta derivar la consulta (o hasta que no se puedan agregar más hechos).

Parte C — Reglas de Inferencia

1. De $(P \Rightarrow Q)$ y P , concluir Q .
Nombre: Modus Ponens. **Válida:** Sí. Si $P \Rightarrow Q$ es verdadera y P es verdadera, la única fila de la tabla de verdad compatible con $P \Rightarrow Q$ verdadera y P verdadera tiene Q verdadera.
2. De $(P \Rightarrow Q)$ y $\neg Q$, concluir $\neg P$.
Nombre: Modus Tollens. **Válida:** Sí. Si P fuera verdadera, $P \Rightarrow Q$ obligaría a Q verdadera, contradiciendo $\neg Q$; por lo tanto P debe ser falsa.
3. De P y Q , concluir $(P \wedge Q)$.
Nombre: Introducción de la conjunción (And-Introduction). **Válida:** Sí, por definición de $\wedge$.
4. De $(P \vee Q)$ y $\neg P$, concluir Q .
Nombre: Silogismo disyuntivo (resolución unitaria). **Válida:** Sí. Como P es falsa, para que $P \vee Q$ sea verdadera, Q debe serlo.

5. De $(P \Rightarrow Q)$ y $(Q \Rightarrow R)$, concluir $(P \Rightarrow R)$.

Nombre: Silogismo hipotético (transitividad de la implicación). **Válida:** Sí (se demuestra formalmente en el Bloque 4, Problema 4.1.3, convirtiendo la sentencia a FNC).

6. De $(P \Rightarrow Q)$ y $\neg P$, concluir $\neg Q$.

Nombre: Negación del antecedente (falacia). **Válida:** No, es **inválida**. **Contraejemplo:** $P = F$, $Q = T$: $P \Rightarrow Q$ es verdadera y $\neg P$ es verdadera, pero $\neg Q$ es falsa.

5.2. Bloque 2: Validez y Satisfacibilidad

Problema 2.1 — Validez y Satisfacibilidad (Ej. 13, Cap. 7 AIMA)

1. $\text{Humo} \Rightarrow \text{Humo}$.

Humo	$\text{Humo} \Rightarrow \text{Humo}$
T	T
F	T

Todas las filas son verdaderas $\Rightarrow$ **Válida** (instancia del esquema $\alpha \Rightarrow \alpha$, siempre tautológico).

2. $\text{Humo} \Rightarrow \text{Fuego}$.

Humo	Fuego	$\text{Humo} \Rightarrow \text{Fuego}$
T	T	T
T	F	F
F	T	T
F	F	T

Hay filas verdaderas y una falsa $\Rightarrow$ **Satisfacible, pero no válida** (contingente).

3. $(\text{Humo} \Rightarrow \text{Fuego}) \Rightarrow (\neg \text{Humo} \Rightarrow \neg \text{Fuego})$.

Humo	Fuego	$H \Rightarrow F$	$\neg H$	$\neg F$	$(H \Rightarrow F) \Rightarrow (\neg H \Rightarrow \neg F)$
T	T	T	F	F	T
T	F	F	F	T	T
F	T	T	T	F	F
F	F	T	T	T	T

La fila $\text{Humo} = F, \text{Fuego} = T$ da Falso, y las demás dan Verdadero $\Rightarrow$ **Satisfacible, pero no válida** (contingente).

4. $\text{Humo} \vee \text{Fuego} \vee \neg \text{Fuego}$.

El subtérmino $\text{Fuego} \vee \neg \text{Fuego}$ ya es una tautología (verdadero sin importar el valor de Fuego), así que toda la disyunción es verdadera en las 4 filas posibles de $(\text{Humo}, \text{Fuego})$ $\Rightarrow$ **Válida**.

5. $((\text{Humo} \wedge \text{Calor}) \Rightarrow \text{Fuego}) \Leftrightarrow ((\text{Humo} \Rightarrow \text{Fuego}) \vee (\text{Calor} \Rightarrow \text{Fuego}))$.

H	C	Fu	$H \wedge C$	$A = (H \wedge C) \Rightarrow Fu$	$H \Rightarrow Fu$	$C \Rightarrow Fu$	B (su OR)	$A \Leftrightarrow B$
T	T	T	T	T	T	T	T	T
T	T	F	T	F	F	F	F	T
T	F	T	F	T	T	T	T	T
T	F	F	F	T	F	T	T	T
F	T	T	F	T	T	T	T	T
F	T	F	F	T	T	F	T	T
F	F	T	F	T	T	T	T	T
F	F	F	F	T	T	T	T	T

Las 8 filas dan Verdadero $\Rightarrow$ **Válida**. (Se puede confirmar algebraicamente: $(H \wedge C) \Rightarrow Fu \equiv \neg H \vee \neg C \vee Fu$, y $(H \Rightarrow Fu) \vee (C \Rightarrow Fu) \equiv (\neg H \vee Fu) \vee (\neg C \vee Fu) \equiv \neg H \vee \neg C \vee Fu$: ambos lados son *idénticos*, por lo que el bicondicional es una tautología.)

Problema 2.2 — Verificación de Consecuencia Lógica

1. $A \Leftrightarrow B \models \neg A \vee B$?

A	B	$A \Leftrightarrow B$	$\neg A \vee B$
T	T	T	T
T	F	F	F
F	T	F	T
F	F	T	T

En las dos filas donde $A \Leftrightarrow B$ es verdadera (fila 1 y fila 4), $\neg A \vee B$ también lo es. **Correcta**. Justificación algebraica: $A \Leftrightarrow B \equiv (A \Rightarrow B) \wedge (B \Rightarrow A)$, y $A \Rightarrow B \equiv \neg A \vee B$; por lo tanto $A \Leftrightarrow B$ implica cada uno de sus conjuntos, en particular $\neg A \vee B$.

2. $(A \vee B) \wedge (\neg C \vee \neg D \vee E) \models (A \vee B)$?

Correcta, y de forma trivial: cualquier modelo que satisface una conjunción $X \wedge Y$ satisface, en particular, cada conjunto por separado (X e Y); aquí $X = (A \vee B)$. No hace falta enumerar las $2^5 = 32$ filas: es una instancia directa de la regla de *eliminación de la conjunción* ($X \wedge Y \models X$), válida para cualquier X, Y .

3. $(A \vee B) \wedge (\neg C \vee \neg D \vee E) \models (A \vee B) \wedge (\neg D \vee E)$?

No es correcta. Contraejemplo: $A = T, B = F, C = F, D = T, E = F$.

Verificación paso a paso:

- $A \vee B = T \vee F = T$.
- $\neg C \vee \neg D \vee E = \neg F \vee \neg T \vee F = T \vee F \vee F = T$.
- Lado izquierdo (premisa) $= T \wedge T = T$: el modelo satisface la premisa.
- $\neg D \vee E = \neg T \vee F = F \vee F = F$.
- Lado derecho (conclusión) $= (A \vee B) \wedge (\neg D \vee E) = T \wedge F = F$.

La premisa es verdadera y la conclusión es falsa en este modelo, así que la premisa **no** implica la conclusión. El error es que $\neg C \vee \neg D \vee E$ puede ser verdadera "gracias a" $\neg C$ sin decir nada sobre $\neg D \vee E$.

5.3. Bloque 3: Modelamiento de Lenguaje Natural — El Problema de la Mansión Embrujada

Parte A — Definición de Símbolos Proposicionales Además de $F_{x,y}, T_{x,y}, Frio_{x,y}$ y $Crujido_{x,y}$ (dados en el enunciado), se necesitan símbolos para registrar el estado epistémico del robot:

- $Segura_{x,y}$ = "El robot ha *probado* que la habitación (x, y) es segura (no tiene Fantasma ni Trampa)".
- $Visitada_{x,y}$ = "El robot ya visitó la habitación (x, y) ".

Parte B — Codificación de las Reglas

1. La entrada es segura:

$$\neg F_{1,1} \wedge \neg T_{1,1}$$

2. Frío en (1, 1) implica Fantasma en una adyacente:

$$Frio_{1,1} \Leftrightarrow (F_{2,1} \vee F_{1,2})$$

3. Crujido en (1, 1) implica Trampa en una adyacente:

$$Crujido_{1,1} \Leftrightarrow (T_{2,1} \vee T_{1,2})$$

4. Axioma de Frío para (2, 2), adyacente a (1, 2), (3, 2) y (2, 1):

$$Frio_{2,2} \Leftrightarrow (F_{1,2} \vee F_{3,2} \vee F_{2,1})$$

5. A lo sumo un Fantasma en toda la mansión (6 habitaciones: (1, 1) a (3, 2)). Usando implicaciones, se afirma que si una habitación tiene el Fantasma, **ninguna otra** lo tiene:

$$\bigwedge_{(x,y) \neq (x',y')} (F_{x,y} \Rightarrow \neg F_{x',y'})$$

Por ejemplo, para (1, 1):

$$F_{1,1} \Rightarrow (\neg F_{2,1} \wedge \neg F_{3,1} \wedge \neg F_{1,2} \wedge \neg F_{2,2} \wedge \neg F_{3,2})$$

y una implicación análoga se escribe partiendo de cada una de las otras 5 habitaciones (en total $\binom{6}{2} = 15$ pares de restricción, cada uno equivalente a la cláusula $\neg F_{x,y} \vee \neg F_{x',y'}$).

Parte C — Razonamiento a partir de las Percepciones Adyacencias en la cuadrícula 3×2 : (1, 1)–(2, 1)–(3, 1) (fila inferior) y (1, 2)–(2, 2)–(3, 2) (fila superior), con (1, 1)–(1, 2), (2, 1)–(2, 2) y (3, 1)–(3, 2) uniendo las dos filas.

1. **¿Puede el robot concluir que (2, 2) contiene una Trampa?**

Razonamiento (paso a paso):

a) En (1, 1) el robot percibe “Sin Crujido”. Por el axioma de la Parte B, ítem 3, $Crujido_{1,1} \Leftrightarrow (T_{2,1} \vee T_{1,2})$; como $Crujido_{1,1}$ es falso, se concluye $\neg T_{2,1} \wedge \neg T_{1,2}$ (ni (2, 1) ni (1, 2) tienen Trampa). Junto con $\neg F_{2,1} \wedge \neg F_{1,2}$ (del axioma de Frío) y la entrada segura, esto confirma que (2, 1) y (1, 2) son seguras — de ahí que el robot pudiera moverse a ambas.

b) El axioma de Crujido para (1, 2) (adyacente a (1, 1) y (2, 2)) es:

$$Crujido_{1,2} \Leftrightarrow (T_{1,1} \vee T_{2,2})$$

c) El robot percibe Crujido en (1, 2), luego $Crujido_{1,2}$ es verdadero, así que $T_{1,1} \vee T_{2,2}$ es verdadero (Modus Ponens sobre el bicondicional).

d) Se sabe que $\neg T_{1,1}$ (la entrada es segura, Parte B ítem 1).

e) Por silogismo disyuntivo sobre $T_{1,1} \vee T_{2,2}$ y $\neg T_{1,1}$, se concluye $T_{2,2}$.

Conclusión: Sí, el robot puede concluir con certeza que hay una Trampa en (2, 2).

2. **¿Puede el robot concluir que (3, 1) es segura para entrar?**

Razonamiento (paso a paso):

a) (3, 1) es adyacente a (2, 1) y (3, 2). El robot solo tiene percepciones en (1, 1), (2, 1) y (1, 2); nunca ha estado en (3, 2), así que no hay información directa sobre esa adyacencia.

- b) En $(2, 1)$ el robot solo percibió Crujido (no Frío). Por el axioma de Frío para $(2, 1)$: $Frio_{2,1} \Leftrightarrow (F_{1,1} \vee F_{3,1} \vee F_{2,2})$. Como $Frio_{2,1}$ es falso, se concluye $\neg F_{1,1} \wedge \neg F_{3,1} \wedge \neg F_{2,2}$: en particular $\neg F_{3,1}$ (no hay Fantasma en $(3, 1)$).
- c) Por el axioma de Crujido para $(2, 1)$: $Crujido_{2,1} \Leftrightarrow (T_{1,1} \vee T_{3,1} \vee T_{2,2})$. Como $Crujido_{2,1}$ es verdadero y $\neg T_{1,1}$ (entrada segura), se obtiene $T_{3,1} \vee T_{2,2}$.
- d) Del ítem anterior de este mismo bloque ya se sabe $T_{2,2}$ es verdadero. Esto por sí solo **ya satisface** la disyunción $T_{3,1} \vee T_{2,2}$, sin necesidad de que $T_{3,1}$ sea verdadero. Es decir, la KB *no fuerza* un valor de verdad para $T_{3,1}$: es consistente tanto con $T_{3,1} = Verdadero$ como con $T_{3,1} = Falso$.

Conclusión: No. Aunque se sabe que $(3, 1)$ no tiene Fantasma ($\neg F_{3,1}$), el estado de la Trampa en $(3, 1)$ queda **indeterminado** (el Crujido percibido en $(2, 1)$ ya queda explicado por la Trampa en $(2, 2)$). Como la seguridad exige certeza sobre *ambos* peligros, el robot no puede probar que $(3, 1)$ sea segura y, según la regla del enunciado, no puede entrar todavía.

5.4. Bloque 4: Conversión a FNC y Resolución

Problema 4.1 — Conversión a FNC

1. $P \Leftrightarrow Q$

Paso a paso:

$$\begin{aligned} P \Leftrightarrow Q &\equiv (P \Rightarrow Q) \wedge (Q \Rightarrow P) && \text{(def. de } \Leftrightarrow \text{)} \\ &\equiv (\neg P \vee Q) \wedge (\neg Q \vee P) && \text{(eliminar } \Rightarrow \text{)} \end{aligned}$$

Ya es conjunción de disyunciones de literales: no hace falta distribuir más.

FNC: $(\neg P \vee Q) \wedge (\neg Q \vee P)$

2. $\neg(P \vee Q) \Rightarrow R$

Paso a paso:

$$\begin{aligned} \neg(P \vee Q) \Rightarrow R &\equiv \neg\neg(P \vee Q) \vee R && \text{(eliminar } \Rightarrow \text{)} \\ &\equiv (P \vee Q) \vee R && \text{(doble negación)} \\ &\equiv P \vee Q \vee R && \text{(ya es una sola cláusula)} \end{aligned}$$

FNC: $P \vee Q \vee R$

3. $(A \Rightarrow B) \wedge (B \Rightarrow C) \Rightarrow (A \Rightarrow C)$

Paso a paso:

$$\begin{aligned} &\equiv \neg[(\neg A \vee B) \wedge (\neg B \vee C)] \vee (\neg A \vee C) && \text{(eliminar las tres } \Rightarrow \text{)} \\ &\equiv [\neg(\neg A \vee B) \vee \neg(\neg B \vee C)] \vee (\neg A \vee C) && \text{(De Morgan)} \\ &\equiv (A \wedge \neg B) \vee (B \wedge \neg C) \vee \neg A \vee C && \text{(De Morgan otra vez, doble negación)} \end{aligned}$$

Distribuir $\vee$ sobre $\wedge$, primero $(A \wedge \neg B) \vee (B \wedge \neg C)$:

$$(A \vee B) \wedge (A \vee \neg C) \wedge (\neg B \vee B) \wedge (\neg B \vee \neg C)$$

Como $\neg B \vee B$ es tautología, se elimina esa cláusula (no aporta restricción):

$$(A \vee B) \wedge (A \vee \neg C) \wedge (\neg B \vee \neg C)$$

Ahora se distribuye ese resultado con $\vee (\neg A \vee C)$, cláusula por cláusula:

$$(A \vee B \vee \neg A \vee C) \wedge (A \vee \neg C \vee \neg A \vee C) \wedge (\neg B \vee \neg C \vee \neg A \vee C)$$

Cada una de las tres cláusulas contiene un literal y su negación ($A, \neg A$ en la primera y segunda; $C, \neg C$ en la segunda y tercera), por lo que **las tres son tautológicas** y se pueden eliminar todas.

Respuesta: La FNC resultante no tiene cláusulas (todas son tautologías), lo cual significa que la sentencia original es verdadera en *todo* modelo: es una **validez** (el silogismo hipotético / transitividad de la implicación, coherente con la Parte C, ítem 5, del Bloque 1).

FNC: $\top$ (conjunción vacía de cláusulas, todas tautológicas)

Problema 4.2 — Validez, FNC y Prueba por Resolución (Ej. 13, Cap. 7 AIMA)

Sentencia: $[(Comida \Rightarrow Fiesta) \vee (Bebidas \Rightarrow Fiesta)] \Rightarrow [(Comida \wedge Bebidas) \Rightarrow Fiesta]$. Se abrevia $Co = Comida$, $Be = Bebidas$, $Fi = Fiesta$.

1. **Clasificación por enumeración:**

Co	Be	Fi	$Co \Rightarrow Fi$	$Be \Rightarrow Fi$	LHS(or)	$Co \wedge Be$	RHS	Total
T	T	T	T	T	T	T	T	T
T	T	F	F	F	F	T	F	T
T	F	T	T	T	T	F	T	T
T	F	F	F	T	T	F	T	T
F	T	T	T	T	T	F	T	T
F	T	F	T	F	T	F	T	T
F	F	T	T	T	T	F	T	T
F	F	F	T	T	T	F	T	T

Clasificación: Válida.

Justificación: Las 8 filas dan verdadero. Es interesante notar que la única fila en la que el consecuente (RHS) es falso es $Co=T, Be=T, Fi=F$; pero en esa misma fila el antecedente (LHS) también es falso, así que la implicación completa es $F \Rightarrow F = T$.

2. **FNC de cada lado:**

FNC lado izquierdo:

$$\begin{aligned} (Co \Rightarrow Fi) \vee (Be \Rightarrow Fi) &\equiv (\neg Co \vee Fi) \vee (\neg Be \vee Fi) && \text{(eliminar } \Rightarrow \text{)} \\ &\equiv \neg Co \vee \neg Be \vee Fi && \text{(ya es una sola cláusula; } Fi \text{ se repite)} \end{aligned}$$

FNC lado izquierdo: $\neg Co \vee \neg Be \vee Fi$

FNC lado derecho:

$$\begin{aligned} (Co \wedge Be) \Rightarrow Fi &\equiv \neg(Co \wedge Be) \vee Fi && \text{(eliminar } \Rightarrow \text{)} \\ &\equiv \neg Co \vee \neg Be \vee Fi && \text{(De Morgan)} \end{aligned}$$

FNC lado derecho: $\neg Co \vee \neg Be \vee Fi$

Confirmación: Ambos lados se reducen exactamente a la *misma* cláusula $\neg Co \vee \neg Be \vee Fi$. Como la sentencia completa tiene la forma $X \Rightarrow X$ (con X idéntico a ambos lados), es una tautología por construcción — esto confirma directamente el resultado de validez obtenido por enumeración en (1), y de forma más eficiente.

3. **Prueba por resolución.** Para probar que la sentencia es válida (es decir, $\models LHS \Rightarrow RHS$), se niega la conclusión y se busca una contradicción entre LHS y $\neg RHS$.

Cláusulas FNC de $(KB \wedge \neg \text{conclusión})$:

- $C_1: \neg Co \vee \neg Be \vee Fi$ (de LHS)
- $\neg RHS = \neg(\neg Co \vee \neg Be \vee Fi) \equiv Co \wedge Be \wedge \neg Fi$ (De Morgan), lo que produce tres cláusulas unitarias:

- $C_2: Co$
- $C_3: Be$
- $C_4: \neg Fi$

Paso de resolución 1: Resolver $C_1 (\neg Co \vee \neg Be \vee Fi)$ con $C_2 (Co)$ sobre el par complementario $Co/\neg Co \Rightarrow$ se deriva $C_5: \neg Be \vee Fi$.

Paso de resolución 2: Resolver $C_5 (\neg Be \vee Fi)$ con $C_3 (Be)$ sobre el par $Be/\neg Be \Rightarrow$ se deriva $C_6: Fi$.

Paso de resolución 3: Resolver $C_6 (Fi)$ con $C_4 (\neg Fi)$ sobre el par $Fi/\neg Fi \Rightarrow$ se deriva la cláusula vacía $\perp$.

Conclusión: Como se derivó $\perp$, el conjunto $\{LHS, \neg RHS\}$ es insatisfacible, por lo tanto $LHS \models RHS$, es decir, la sentencia original es **válida**. Esto confirma, por tercera vía, el resultado de (1) y (2).

5.5. Bloque 5: Cláusulas de Horn, Cláusulas Definidas y Encadenamiento

Parte A — Clasificación de Cláusulas Sección A.1

#	Cláusula	Clasificación (justificación)
1	$A \vee \neg B \vee \neg C$	Definida, Horn (1 literal positivo: A)
2	$\neg A \vee \neg B$	Horn, Objetivo (0 literales positivos)
3	A	Definida, Horn, Unitaria (1 literal, positivo)
4	$A \vee B \vee \neg C$	No-Horn (2 literales positivos: A, B)
5	$\neg A \vee \neg B \vee \neg C$	Horn, Objetivo (0 literales positivos)
6	$B \wedge C \Rightarrow A \equiv \neg B \vee \neg C \vee A$	Definida, Horn (1 positivo: A)
7	$Verdadero \Rightarrow A \equiv A$	Definida, Horn, Unitaria (equivalente a la cláusula 3)

Sección A.2 — Clasificación desde lenguaje natural (Ej. 17, Cap. 7)

Enunciado: “Una persona que es radical (R) es elegible (E) si es conservadora (C), pero de lo contrario no es elegible.” Esto se lee como: *si* R , entonces (E sii C); la oración no afirma nada sobre las personas no radicales.

1. Análisis de las tres opciones:

- **Opción 1** ($(R \wedge E) \Leftrightarrow C$): **Incorrecta.** Fuerza C a ser falso cada vez que R es falso (porque si $R=F$, entonces $R \wedge E = F$, y el bicondicional exige $C = F$), es decir, afirma que ninguna persona no-radical puede ser conservadora — algo que el enunciado original nunca dice.
- **Opción 2** ($R \Rightarrow (E \Leftrightarrow C)$): **Correcta.** Traduce exactamente “si radical, entonces elegible sii conservador”, y es vacuamente verdadera (no afirma nada) cuando R es falso, tal como exige el enunciado.
- **Opción 3** ($R \Rightarrow ((C \Rightarrow E) \vee \neg E)$): **Incorrecta.** Simplificando el consecuente: $(C \Rightarrow E) \vee \neg E \equiv (\neg C \vee E) \vee \neg E \equiv \neg C \vee (E \vee \neg E) \equiv \neg C \vee \top \equiv \top$. El consecuente es una tautología, así que toda la fórmula se reduce a $R \Rightarrow \top \equiv \top$: es siempre verdadera y no expresa ninguna restricción real.

2. Conversión a forma clausular de la opción correcta (Opción 2):

$$\begin{aligned}
 R \Rightarrow (E \Leftrightarrow C) &\equiv \neg R \vee (E \Leftrightarrow C) \\
 &\equiv \neg R \vee [(\neg E \vee C) \wedge (\neg C \vee E)] && \text{(bicondicional)} \\
 &\equiv (\neg R \vee \neg E \vee C) \wedge (\neg R \vee \neg C \vee E) && \text{(distribuir } \vee \text{ sobre } \wedge)
 \end{aligned}$$

Respuesta: Dos cláusulas: $(\neg R \vee \neg E \vee C)$ y $(\neg R \vee \neg C \vee E)$.

Respuesta (Horn): Sí, ambas son cláusulas de Horn. La primera tiene exactamente

un literal positivo (C), la segunda tiene exactamente un literal positivo (E); ambas son de hecho **definidas** ($R \wedge E \Rightarrow C$ y $R \wedge C \Rightarrow E$ respectivamente).

3. ¿Podría un algoritmo de encadenamiento resolver esta KB?

Respuesta: Sí. Como la representación correcta (Opción 2) se descompone en dos cláusulas definidas (de Horn), tanto el encadenamiento hacia adelante como el encadenamiento hacia atrás son correctos y completos para procesarla, siempre que el resto de la KB (hechos sobre R , C , E para individuos concretos) también esté en forma de Horn.

Parte B — Encadenamiento Hacia Adelante KB: $H_1 : A$, $H_2 : B$, $R_1 : A \wedge B \Rightarrow C$, $R_2 : C \Rightarrow D$, $R_3 : B \wedge D \Rightarrow E$, $R_4 : E \Rightarrow F$, $R_5 : A \wedge F \Rightarrow G$.

1. Traza:

Paso	Regla Aplicada	Nuevo Hecho Derivado
1	H_1 (hecho inicial)	A
2	H_2 (hecho inicial)	B
3	$R_1: A \wedge B \Rightarrow C$ (A, B ya conocidos)	C
4	$R_2: C \Rightarrow D$ (C ya conocido)	D
5	$R_3: B \wedge D \Rightarrow E$ (B, D ya conocidos)	E
6	$R_4: E \Rightarrow F$ (E ya conocido)	F
7	$R_5: A \wedge F \Rightarrow G$ (A, F ya conocidos)	G

- ¿Es G consecuencia lógica? **Sí.** El encadenamiento hacia adelante es correcto (sound) y completo para KB de Horn; como derivó G en el paso 7 mediante una cadena de reglas cuyas premisas estaban satisfechas en cada momento, se concluye $KB \models G$.
- Literales que son consecuencia lógica de la KB:** A, B, C, D, E, F, G — todos los hechos derivados por la traza, ya que el algoritmo terminó sin poder agregar nada más y ningún otro símbolo aparece en la KB.

Parte C — Encadenamiento Hacia Atrás

1. Árbol de prueba Y-O para la consulta G :

G	(meta)
'-- R5: A y F => G	(nodo-Y: se requieren ambas ramas)
-- A	(hecho, hoja OK)
'-- F	
'-- R4: E => F	
'-- E	
'-- R3: B y D => E	(nodo-Y: se requieren ambas ramas)
-- B	(hecho, hoja OK)
'-- D	
'-- R2: C => D	
'-- C	
'-- R1: A y B => C	(nodo-Y: se requieren ambas ramas)
-- A	(hecho, hoja OK, ya probado)
'-- B	(hecho, hoja OK, ya probado)

Cada nodo etiquetado con una regla es un nodo “Y” (AND): todas sus ramas hijas deben probarse. La recursión termina exitosamente cuando cada hoja es un hecho de la KB (A o B).

2. Comparación:

Pasos hacia adelante: 7 hechos derivados (incluye los 2 hechos iniciales), en un único recorrido que expande *toda* la KB sin importar la consulta.

Pasos hacia atrás: Se exploran únicamente las submetas necesarias para G : A, F, E, B, D, C (mismos 7 nodos en este caso particular, porque esta KB es una cadena lineal donde cada hecho es relevante para G), pero solo se expanden las reglas cuya *cabeza* coincide con la meta actual, nunca reglas irrelevantes.

Preferencia: En esta KB lineal ambos métodos exploran esencialmente el mismo número de nodos, así que no hay ventaja clara. En general, se prefiere **hacia adelante** cuando se quieren derivar *todas* las consecuencias de la KB (o la KB es pequeña y densa en reglas relevantes), y **hacia atrás** cuando solo interesa responder una consulta puntual en una KB grande con muchos hechos/reglas irrelevantes para esa consulta (evita trabajo innecesario).

Parte D — Completitud y Limitaciones

1. ¿Puede el encadenamiento hacia adelante manejar $(A \vee B \Rightarrow C)$?

Respuesta: No directamente en esa forma sintáctica, porque el algoritmo espera premisas unidas por $\wedge$ (todas deben cumplirse a la vez), no por $\vee$. Sin embargo, al convertir la cláusula a FNC se obtiene $(A \vee B) \Rightarrow C \equiv \neg(A \vee B) \vee C \equiv (\neg A \wedge \neg B) \vee C \equiv (\neg A \vee C) \wedge (\neg B \vee C)$, es decir, dos cláusulas de Horn separadas y equivalentes: $A \Rightarrow C$ y $B \Rightarrow C$. Una vez descompuesta así, el encadenamiento hacia adelante sí puede procesarla (disparando C si se conoce A o si se conoce B , por separado).

2. KB con solo cláusulas no-Horn — ¿qué algoritmos sirven?

Respuesta: Al menos dos: (1) **TT-Implica?** (enumeración de tablas de verdad), correcto y completo para cualquier KB proposicional aunque exponencial; (2) **Refutación por resolución**, correcta y completa para cualquier conjunto de cláusulas en FNC, sin requerir estructura de Horn. (También podría usarse DPLL para decidir la satisfacibilidad de $KB \wedge \neg\alpha$.)

3. Complejidad temporal: encadenamiento hacia adelante (Horn) vs. TT-Implica? (general).

Respuesta: El encadenamiento hacia adelante sobre una KB de Horn es **lineal** en el tamaño de la KB, $\mathcal{O}(n)$: cada cláusula se examina y dispara a lo sumo una vez, llevando un contador de premisas pendientes por cláusula, sin necesidad de retroceder ni de "adivinar" valores de verdad. TT-Implica? sobre una KB proposicional general es **exponencial**, $\mathcal{O}(2^n)$ en el número de símbolos, porque enumera todas las asignaciones de verdad posibles sin asumir ninguna estructura.

Respuesta (por qué): La diferencia se debe a que las cláusulas de Horn (a lo sumo un literal positivo) tienen una estructura dirigida (premisas $\Rightarrow$ conclusión) que permite propagar hechos ya establecidos de forma monótona y determinista, sin bifurcaciones; una KB general (con cláusulas no-Horn) no tiene esa estructura, y en el peor caso hay que probar combinaciones de valores de verdad para decidir la consecuencia lógica.

6. Ejercicios: Guía de Lógica de Primer Orden (con solución paso a paso)

Esta sección resuelve, de principio a fin, la *Guía de Ejercicios: Lógica de Primer Orden*.

6.1. Bloque 1: Conceptos Fundamentales de LPO

1. "Una sentencia válida en LPO es verdadera en toda interpretación posible, independientemente de la asignación a variables libres."

Verdadero. Por definición, una **sentencia** (a diferencia de una fórmula abierta) no tiene variables libres, así que la cláusula “independientemente de la asignación a variables libres” se cumple trivialmente (no hay ninguna que asignar); lo que exige la validez es que sea verdadera en *todo* modelo/interpretación posible.

2. “Si una fórmula Φ es satisfacible, entonces su negación ($\neg\Phi$) debe ser insatisfacible.”

Falso. Satisfacibilidad de Φ solo significa que Φ es verdadera en *al menos un* modelo; nada impide que también exista otro modelo donde Φ (y por lo tanto $\neg\Phi$) sea verdadera.

Contraejemplo: $\Phi = P(x)$. Es satisfacible (verdadera en una interpretación donde P es siempre verdadero). Pero $\neg\Phi = \neg P(x)$ también es satisfacible (verdadera en una interpretación donde P es siempre falso): no es insatisfacible.

Corrección: Solo si Φ es **válida** se garantiza que $\neg\Phi$ es insatisfacible.

3. “La sentencia $\forall x P(x) \rightarrow \exists x P(x)$ es válida en LPO, siempre que el dominio sea no vacío.”

Verdadero. Si el dominio no es vacío, existe al menos un objeto d ; si $\forall x P(x)$ es verdadera, en particular $P(d)$ es verdadera, lo cual basta para que $\exists x P(x)$ sea verdadera. (Precisamente por esto LPO exige dominios no vacíos: con dominio vacío, $\forall x P(x)$ sería vacuamente verdadera mientras que $\exists x P(x)$ sería falsa, y la implicación fallaría.)

4. “Las fórmulas $\forall x \exists y R(x, y)$ y $\exists y \forall x R(x, y)$ son lógicamente equivalentes.”

Falso. Solo se cumple una dirección: $\exists y \forall x R(x, y) \models \forall x \exists y R(x, y)$ (si hay un y que sirve para todo x , ese mismo y sirve como testigo para cada x por separado), pero no la recíproca.

Contraejemplo: Dominio = enteros, $R(x, y) \equiv “x < y”$. $\forall x \exists y (x < y)$ es verdadera (para todo entero siempre hay uno mayor). Pero $\exists y \forall x (x < y)$ es falsa (ningún entero es mayor que *todos* los enteros).

5. “La fórmula $\forall x(\Phi \vee \Psi)$ es lógicamente equivalente a $(\forall x \Phi \vee \forall x \Psi)$ para cualesquiera Φ y Ψ .”

Falso. Solo se cumple una dirección: $(\forall x \Phi \vee \forall x \Psi) \models \forall x(\Phi \vee \Psi)$, pero no la recíproca (a diferencia de $\wedge$, que sí distribuye en ambas direcciones sobre $\forall$).

Contraejemplo: Dominio = $\{1, 2\}$, $\Phi(x) = “x$ es impar” (verdadero solo para $x=1$), $\Psi(x) = “x$ es par” (verdadero solo para $x=2$). $\forall x(\Phi \vee \Psi)$ es verdadera (todo elemento es par o impar), pero $\forall x \Phi$ es falsa (falla en $x=2$) y $\forall x \Psi$ es falsa (falla en $x=1$), así que $\forall x \Phi \vee \forall x \Psi$ es falsa.

6.2. Bloque 2: Traducción — Español a Lógica de Primer Orden

Vocabulario dado: $Ocupacion(p, o)$, $Cliente(p_1, p_2)$ (“ p_1 es cliente de p_2 ”), $Jefe(p_1, p_2)$, constantes de ocupación *Medico*, *Cirujano*, *Abogado*, *Actor*, *Mecanico*, constantes *Emily*, *Joe*, *Enfermero(p)*, $LeAgrad(p_1, p_2)$ (“a p_1 le agrada p_2 ”).

1. “Emily es cirujana o abogada.”

$$Ocupacion(Emily, Cirujano) \vee Ocupacion(Emily, Abogado)$$

2. “Joe es actor, pero también ejerce otra ocupación.”

$$Ocupacion(Joe, Actor) \wedge \exists o (Ocupacion(Joe, o) \wedge o \neq Actor)$$

Justificación: “otra” ocupación requiere afirmar la *existencia* de una ocupación distinta de *Actor*; por eso el cuantificador existencial se combina con $\wedge$ (existe un objeto que cumple ser ocupación de Joe **y** ser distinto de Actor), y se necesita explícitamente la desigualdad $o \neq Actor$, si no la misma ocupación *Actor* satisfaría trivialmente el existencial.

3. “Todos los cirujanos son médicos.”

$$\forall p (Ocupacion(p, Cirujano) \Rightarrow Ocupacion(p, Medico))$$

Justificación: Enunciado universal sobre una categoría $\Rightarrow$ se usa $\forall$ con $\Rightarrow$, tal como sugiere el consejo de la guía.

4. “Existe un abogado todos cuyos clientes son médicos.”

$$\exists p_1 (Ocupacion(p_1, Abogado) \wedge \forall p_2 (Cliente(p_2, p_1) \Rightarrow Ocupacion(p_2, Medico)))$$

Justificación: El cuantificador principal es existencial (“existe un abogado”), así que se combina con $\wedge$ para afirmar sus dos propiedades (ser abogado, y la condición sobre sus clientes). Dentro, “todos sus clientes” es un $\forall$ interno con $\Rightarrow$. Nótese el orden de argumentos: $Cliente(p_2, p_1)$ significa “ p_2 es cliente de p_1 ”, que es lo que se necesita (clientes *del abogado* p_1).

5. “Algún mecánico les agrada a todos los enfermeros.”

$$\exists p_1 (Ocupacion(p_1, Mecanico) \wedge \forall p_2 (Enfermero(p_2) \Rightarrow LeAgrada(p_2, p_1)))$$

Justificación: De nuevo $\exists$ con $\wedge$ para el mecánico, y $\forall$ con $\Rightarrow$ para “todos los enfermeros”. Cuidado con el orden de *LeAgrada*: como $LeAgrada(p_1, p_2)$ significa “a p_1 le agrada p_2 ”, y se quiere decir que *a los enfermeros les agrada el mecánico*, se escribe $LeAgrada(p_2, p_1)$ (enfermero, mecánico), no al revés.

6.3. Bloque 3: LPO a Español y Análisis de Validez

Parte A — Traducción al español

$$1. \forall x, y, l \text{ HablaIdioma}(x, l) \wedge \text{HablaIdioma}(y, l) \Rightarrow \text{Entiende}(x, y) \wedge \text{Entiende}(y, x)$$

Español: “Si dos personas hablan el mismo idioma, entonces se entienden mutuamente (cada una entiende a la otra).”

$$2. \exists y \forall x \text{ Entiende}(x, y)$$

Español: “Existe una persona a quien todo el mundo entiende.”

$$3. \forall x \exists y (\text{Amigos}(x, y) \wedge \forall z (\text{Amigos}(x, z) \rightarrow \text{Entiende}(z, y)))$$

Español: “Toda persona tiene un amigo tal que *todos* los amigos de esa persona entienden a ese amigo en particular.”

Parte B — Validez, satisfacibilidad, insatisfacibilidad

$$4. (\exists x x=x) \Rightarrow (\forall y \exists z z \neq y)$$

Clasificación: Satisfacible, pero no válida (contingente).

Justificación: El antecedente $\exists x x=x$ es verdadero en *todo* modelo (por reflexividad de la igualdad, y porque los dominios en LPO siempre son no vacíos), así que la verdad de la implicación depende enteramente del consecuente $\forall y \exists z z \neq y$ (“todo elemento tiene otro distinto de él”), que es verdadero en cualquier dominio con **2 o más** elementos, pero **falso** en un dominio de un solo elemento $D = \{a\}$ (para $y = a$ no existe ningún $z \in D$ con $z \neq a$).

Con $|D| \geq 2$: antecedente T , consecuente $T \Rightarrow$ implicación T (satisfacible).

Con $|D| = 1$: antecedente T , consecuente $F \Rightarrow$ implicación F (no válida).

5. $\forall x (P(x) \vee \neg P(x))$

Clasificación: Válida.

Justificación: Para cualquier interpretación de P y cualquier objeto del dominio, $P(x) \vee \neg P(x)$ es una instancia del principio del tercero excluido, verdadera sin importar el valor de verdad de $P(x)$. Al ser verdadera para todo x en todo modelo, la sentencia universal también lo es.

6. $\forall x (P(x) \wedge Q(x)) \leftrightarrow (\forall x P(x) \wedge \forall x Q(x))$

Clasificación: Válida.

Justificación: A diferencia de $\forall/\exists$, el cuantificador universal **sí distribuye completamente** sobre $\wedge$ en ambas direcciones: “todo objeto cumple P y Q ” equivale exactamente a “todo objeto cumple P ” y “todo objeto cumple Q ”, para cualquier interpretación de P, Q y cualquier dominio — no existe contraejemplo posible.

7. $\forall x (P(x) \vee Q(x)) \leftrightarrow (\forall x P(x) \vee \forall x Q(x))$

Clasificación: Satisfacible, pero no válida (contingente).

Justificación: Solo la dirección $(\forall x P(x) \vee \forall x Q(x)) \models \forall x (P(x) \vee Q(x))$ es válida en general; la recíproca falla.

Contraejemplo (para mostrar que no es válida): Dominio $\{1, 2\}$, $P(x) = “x \text{ impar}”$, $Q(x) = “x \text{ par}”$. $\forall x (P \vee Q)$ es verdadero, pero $\forall x P$ y $\forall x Q$ son ambos falsos, así que el lado derecho es falso: el bicondicional es falso en este modelo (no es válida).

Modelo donde es verdadera (para mostrar que es satisfacible, no insatisfacible): Cualquier interpretación donde $P(x)$ sea verdadero para todo x (p. ej. $P(x) \equiv (x=x)$): ambos lados del bicondicional son verdaderos, así que el bicondicional completo es verdadero.

6.4. Bloque 4: Construcción de Base de Conocimiento — Dominio de Parentesco

Predicados base: $Padre(p, q)$ (“ p es progenitor de q ”), $Hombre(p)$, $Mujer(p)$, $Casado(p, q)$.

Parte A — Axiomas

1. **Madre**(m, h): m es la madre de h .

$$Madre(m, h) \Leftrightarrow Padre(m, h) \wedge Mujer(m)$$

2. **PadreBio**(f, h): f es el padre biológico de h .

$$PadreBio(f, h) \Leftrightarrow Padre(f, h) \wedge Hombre(f)$$

3. **Hermano**(x, y): x e y son hermanos (comparten al menos un progenitor, $x \neq y$).

$$Hermano(x, y) \Leftrightarrow x \neq y \wedge \exists p (Padre(p, x) \wedge Padre(p, y))$$

4. **Abuelo**(g, h): g es abuelo o abuela de h .

$$Abuelo(g, h) \Leftrightarrow \exists p (Padre(g, p) \wedge Padre(p, h))$$

5. **Abuela**(g, h): g es abuela de h .

$$Abuela(g, h) \Leftrightarrow Abuelo(g, h) \wedge Mujer(g)$$

Parte B — Hechos del árbol genealógico

Casado(Jorge, Mama)

Padre(Jorge, Isabel), Padre(Jorge, Margarita), Padre(Mama, Isabel), Padre(Mama, Margarita)
Padre(Isabel, Carlos), Padre(Isabel, Ana), Padre(Isabel, Andres), Padre(Isabel, Eduardo)
Padre(Felipe, Carlos), Padre(Felipe, Ana), Padre(Felipe, Andres), Padre(Felipe, Eduardo)
Padre(Carlos, Guillermo), Padre(Carlos, Enrique), Padre(Diana, Guillermo), Padre(Diana, Enrique)
Padre(Ana, Pedro), Padre(Ana, Zara), Padre(Marcos, Pedro), Padre(Marcos, Zara)

Hombre(Jorge), Hombre(Felipe), Hombre(Carlos), Hombre(Marcos), Hombre(Andres),
Hombre(Eduardo), Hombre(Guillermo), Hombre(Enrique), Hombre(Pedro)

Mujer(Mama), Mujer(Isabel), Mujer(Margarita), Mujer(Diana), Mujer(Ana), Mujer(Zara)

Nota: Solo se codifica *Casado(Jorge, Mama)* porque es el único matrimonio que el enunciado afirma explícitamente; no se asumen matrimonios no declarados (p. ej. Isabel-Felipe).

Parte C — Consultas

1. ¿Quiénes son los nietos de Isabel?

Se instancia el axioma $Abuelo(g, h) \Leftrightarrow \exists p(Padre(g, p) \wedge Padre(p, h))$ con $g = Isabel$. Los hijos de Isabel son $\{Carlos, Ana, Andres, Eduardo\}$ (progenitor directo); se revisa cuáles de ellos tienen hijos propios en la KB:

- $Padre(Isabel, Carlos) \wedge Padre(Carlos, Guillermo) \Rightarrow Abuelo(Isabel, Guillermo)$
- $Padre(Isabel, Carlos) \wedge Padre(Carlos, Enrique) \Rightarrow Abuelo(Isabel, Enrique)$
- $Padre(Isabel, Ana) \wedge Padre(Ana, Pedro) \Rightarrow Abuelo(Isabel, Pedro)$
- $Padre(Isabel, Ana) \wedge Padre(Ana, Zara) \Rightarrow Abuelo(Isabel, Zara)$
- *Andres y Eduardo no tienen hijos registrados en la KB, así que no aportan nietos.*

Respuesta: Los nietos de Isabel son **Guillermo, Enrique, Pedro y Zara.**

2. ¿Son Guillermo y Zara hermanos?

Se evalúa $Hermano(Guillermo, Zara) \Leftrightarrow Guillermo \neq Zara \wedge \exists p(Padre(p, Guillermo) \wedge Padre(p, Zara))$.

Progenitores de Guillermo: $\{Carlos, Diana\}$. Progenitores de Zara: $\{Ana, Marcos\}$. La intersección $\{Carlos, Diana\} \cap \{Ana, Marcos\} = \emptyset$: no existe ningún p que sea progenitor de ambos.

Respuesta: No son hermanos; el existencial del axioma de Hermano queda insatisfecho, así que $Hermano(Guillermo, Zara)$ es falso. (De hecho son **primos**: sus respectivos padres, Carlos y Ana, sí son hermanos entre sí, pues ambos son hijos de Isabel y Felipe.)

6.5. Bloque 5: Unificación y Demostración por Resolución

Parte A — Unificación

1. $P(A, B, B)$ y $P(x, y, z)$.

Posición a posición: $x/A, y/B, z/B$.

UMG: $\theta = \{x/A, y/B, z/B\}$.

2. $Q(y, G(A, B))$ y $Q(G(x, x), y)$.

Posición 1: y vs. $G(x, x) \Rightarrow \theta_1 = \{y/G(x, x)\}$ (verificación de ocurrencia OK: y no aparece dentro de $G(x, x)$).

Posición 2 (aplicando θ_1 a ambos términos): $G(A, B)$ vs. $G(x, x) \Rightarrow$ se debe unificar A con x (da x/A) y, en la segunda posición de G , B con x (ya sustituido por A), es decir B vs. A . Como A y B son constantes *distintas*, esta comparación falla.

Respuesta: No se pueden unificar. El intento exige simultáneamente x/A y x/B , lo cual es imposible porque $A \neq B$ (una variable no puede ligarse a dos constantes distintas a la vez).

3. $LeAgrada(x, y)$ y $LeAgrada(Joe, z)$.

Posición 1: x vs. $Joe \Rightarrow x/Joe$.

Posición 2: y vs. z (ambas variables) $\Rightarrow y/z$.

UMG: $\theta = \{x/Joe, y/z\}$ (se deja y ligada a la variable z , no a una constante, para mantener el unificador lo más general posible).

4. $Progenitor(Padre(x), x)$ y $Progenitor(Padre(Juan), Juan)$.

Posición 1: $Padre(x)$ vs. $Padre(Juan) \Rightarrow$ se unifican los argumentos internos: x vs. $Juan \Rightarrow x/Juan$.

Posición 2 (aplicando $x/Juan$): x vs. $Juan \Rightarrow Juan$ vs. $Juan$: coincide, sin generar nueva ligadura.

UMG: $\theta = \{x/Juan\}$.

Parte B — Conversión a FNC

5. $\forall x (Caballo(x) \Rightarrow Animal(x))$

Paso a paso:

$$\forall x (Caballo(x) \Rightarrow Animal(x)) \equiv \forall x (\neg Caballo(x) \vee Animal(x)) \quad (\text{eliminar } \Rightarrow)$$

No hay $\exists$ que skolemizar. Se elimina el cuantificador universal (queda implícito sobre todas las variables libres de la cláusula) y ya es una disyunción de literales.

FNC: $\neg Caballo(x) \vee Animal(x)$

6. $\forall h ((\exists y (CabezaDe(h, y) \wedge Caballo(y))) \Rightarrow (\exists z (CabezaDe(h, z) \wedge Animal(z))))$

Paso 1 (eliminar $\Rightarrow$):

$$\forall h (\neg (\exists y (CabezaDe(h, y) \wedge Caballo(y))) \vee \exists z (CabezaDe(h, z) \wedge Animal(z)))$$

Paso 2 (mover $\neg$ hacia adentro; $\neg \exists y \Phi \equiv \forall y \neg \Phi$, De Morgan):

$$\forall h (\forall y (\neg CabezaDe(h, y) \vee \neg Caballo(y)) \vee \exists z (CabezaDe(h, z) \wedge Animal(z)))$$

Paso 3 (estandarizar variables): h, y, z ya son distintas, no hace falta renombrar.

Paso 4 (Skolemización): El $\exists z$ está dentro del alcance de $\forall h$ únicamente (no dentro de $\forall y$, que solo gobierna el primer disyunto), así que se reemplaza z por una **función de Skolem** de h : $z \rightarrow S(h)$.

$$\forall h \forall y ((\neg CabezaDe(h, y) \vee \neg Caballo(y)) \vee (CabezaDe(h, S(h)) \wedge Animal(S(h))))$$

Paso 5 (eliminar cuantificadores universales, quedan implícitos):

$$(\neg CabezaDe(h, y) \vee \neg Caballo(y)) \vee (CabezaDe(h, S(h)) \wedge Animal(S(h)))$$

Paso 6 (distribuir $\vee$ sobre $\wedge$): con $A_1 = \neg CabezaDe(h, y)$, $A_2 = \neg Caballo(y)$, $B_1 = CabezaDe(h, S(h))$, $B_2 = Animal(S(h))$:

$$(A_1 \vee A_2 \vee B_1) \wedge (A_1 \vee A_2 \vee B_2)$$

FNC (dos cláusulas):

$$\neg CabezaDe(h, y) \vee \neg Caballo(y) \vee CabezaDe(h, S(h)) \quad (\text{C-A})$$

$$\neg CabezaDe(h, y) \vee \neg Caballo(y) \vee Animal(S(h)) \quad (\text{C-B})$$

Parte C — Refutación por Resolución **Objetivo:** demostrar que la premisa 5 (“todo caballo es un animal”) implica la afirmación 6 (“la cabeza de un caballo es la cabeza de un animal”), por refutación: se asume la **negación** de la conclusión (sentencia 6), se convierte a FNC, y se resuelve junto con la premisa (sentencia 5) hasta obtener $\perp$.

Paso 1 — Cláusula de la premisa (de la Parte B, ítem 5):

$$C_1 : \neg Caballo(x) \vee Animal(x)$$

Paso 2 — Negar la conclusión (sentencia 6) y convertirla a FNC:

$$\begin{aligned} \neg \left[\forall h (\exists y (CabezaDe(h, y) \wedge Caballo(y)) \Rightarrow \exists z (CabezaDe(h, z) \wedge Animal(z))) \right] &\equiv \exists h \left[\exists y (CabezaDe(h, y) \wedge Caballo(y)) \wedge \neg \exists z (CabezaDe(h, z) \wedge Animal(z)) \right] \\ &\equiv \exists h \exists y \left[CabezaDe(h, y) \wedge Caballo(y) \wedge \neg \exists z (CabezaDe(h, z) \wedge Animal(z)) \right] \\ &\equiv \exists h \exists y \left[CabezaDe(h, y) \wedge Caballo(y) \wedge \forall z (\neg CabezaDe(h, z) \vee \neg Animal(z)) \right] \end{aligned}$$

Los cuantificadores $\exists h$ y $\exists y$ son los **más externos** (no están dentro de ningún $\forall$), así que se Skolemizan con **constantes** nuevas H (una cabeza concreta) y Y (un caballo concreto): $h \rightarrow H$, $y \rightarrow Y$. El $\forall z$ se elimina (queda implícito) y la conjunción se separa en cláusulas:

$$C_2 : CabezaDe(H, Y)$$

$$C_3 : Caballo(Y)$$

$$C_4 : \neg CabezaDe(H, z) \vee \neg Animal(z)$$

Paso 3 — Conjunto de cláusulas para la refutación: $\{C_1, C_2, C_3, C_4\}$.

Paso de resolución 1: Resolver C_1 ($\neg Caballo(x) \vee Animal(x)$) con C_3 ($Caballo(Y)$), unificador $\theta = \{x/Y\}$, sobre el par $Caballo(Y)/\neg Caballo(x) \Rightarrow$ se deriva C_5 : $Animal(Y)$.

Paso de resolución 2: Resolver C_4 ($\neg CabezaDe(H, z) \vee \neg Animal(z)$) con C_2 ($CabezaDe(H, Y)$), unificador $\theta = \{z/Y\}$, sobre el par $CabezaDe(H, Y)/\neg CabezaDe(H, z) \Rightarrow$ se deriva C_6 : $\neg Animal(Y)$.

Paso de resolución 3: Resolver C_5 ($Animal(Y)$) con C_6 ($\neg Animal(Y)$), sin unificador adicional (ambos ya están en términos de la constante Y) $\Rightarrow$ se deriva la **cláusula vacía** $\perp$.

Conclusión: Se derivó $\perp$ a partir de {premisa, $\neg$ conclusión}, por lo tanto ese conjunto es insatisfacible. Por refutación, esto confirma que la premisa **implica** la conclusión: la cabeza de un caballo es, en efecto, la cabeza de un animal.

Parte D — Reflexión

7. ¿Por qué la resolución funciona por contradicción?

Para demostrar $BC \models \alpha$ semánticamente habría que revisar *todo* modelo de BC y confirmar que α es verdadera en cada uno — inviable en general en LPO (dominios infinitos). La refutación invierte el problema: asume $\neg\alpha$ y pregunta si $BC \wedge \neg\alpha$ tiene *algún* modelo. Si se logra derivar $\perp$ (la cláusula vacía, sintácticamente insatisfacible), entonces $BC \wedge \neg\alpha$ no tiene ningún modelo, es decir, en **ningún** modelo pueden ser simultáneamente verdaderas BC y $\neg\alpha$. Eso equivale exactamente a decir que en todo modelo donde BC es verdadera, α también lo es — la definición misma de $BC \models \alpha$. La resolución convierte así una pregunta sobre *todos los modelos* (semántica) en una búsqueda de una prueba sintáctica finita de inconsistencia, lo cual es mecánicamente verificable.

8. ¿Por qué la resolución en LPO es solo semi-decidible?

En lógica proposicional el número de símbolos es finito, así que hay exactamente 2^n modelos posibles: TT-Implica? siempre termina (decidible). En LPO los dominios de cuantificación pueden ser infinitos, y la resolución con unificación puede necesitar instanciar variables

universales de infinitas maneras distintas, generando una secuencia potencialmente infinita de cláusulas nuevas. El **teorema de completitud** garantiza que si $BC \models \alpha$, la resolución *sí* encontrará $\perp$ en un número **finito** de pasos, tarde o temprano. Pero si $BC \not\models \alpha$, no existe garantía de terminación: el algoritmo puede seguir generando cláusulas para siempre sin nunca confirmar que α *no* se sigue. Por eso el procedimiento es **semi-decidible**: puede confirmar un “sí” en tiempo finito siempre que la respuesta sea sí, pero no puede garantizar confirmar un “no” en tiempo finito.